are-self: A Revolutionary Agentic Brain for Graph-Based Orchestration and Autonomous Reasoning

are-self Project

March 2026

Abstract

are-self is a Django control plane that couples two systems: (A) a graph-based orchestration engine that launches and monitors build or automation steps across the local machine and remote agents; and (B) an autonomous reasoning engine that staffs identity-driven AI workers, gives them tools, stores their memory, and lets them manipulate an Agile work graph. The system is built around persistent, typed state rather than transient in-memory orchestration. The orchestration side executes labeled directed graphs of Neurons and Axons as waves of Spikes, driven by Celery tasks, with a formal transition rule $L_{\text{enabled}}(s)$ over edge labels $\{\text{flow}, \text{success}, \text{failure}\}$. The cognition side implements a turn-based reasoning loop with a game-like focus/XP economy, vector-based long-term memory (Hippocampus), and a work-allocation gatekeeper (Prefrontal Cortex) that decides whether an AI worker has actionable work before creating a session. This document presents the main points and breakthroughs of are-self, as seen in its codebase [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13].

1 Introduction

The development of AI-assisted automation platforms has reached a point where orchestration and reasoning must be tightly integrated. are-self addresses this by providing a single Django application that unifies graph-based workflow execution with identity-driven AI cognition. The live runtime is the `central_nervous_system` stack plus brain-themed apps: frontal_lobe (reasoning), parietal_lobe (tools), hippocampus (memory), temporal_lobe (scheduler), prefrontal_cortex (work allocation), identity (persona), synaptic_cleft (websocket bus), and occipital_lobe (log digestion) [1].

Contributions.

- We describe are-self as a persisted state machine platform where the CNS executes external work as graph waves and the frontal/parietal/hippocampus/temporal stack executes internal cognitive work as turn waves [1].
- We present the CNS graph model $G = (V, E, \lambda)$ with a formal transition rule for wave dispatch, blackboard propagation, and hierarchical subgraph composition [3, 2].
- We formalize the reasoning loop architecture: identity-driven prompts, L1/L2 history cache, focus/XP economy, tool fizzle condition, and novelty-based memory rewards [4, 5, 6].
- We document the temporal scheduler and Prefrontal Cortex work-eligibility gate: Temporal Lobe decides *when* an AI could work; Prefrontal Cortex decides *whether* it should work [7, 8].
- We summarize the unified mathematical structure across modules: context resolution precedence, economy systems, and state machines [2].

2 Background and Problem Statement

2.1 Orchestration and cognition as separate concerns

Traditional automation platforms treat workflow orchestration and AI reasoning as separate domains. are-self unifies them: a reasoning run is another kind of spike execution, invoked via the native handler `run_frontal_lobe`. The same graph vocabulary (SpikeTrain, Spike, Effector) applies to both process execution and LLM turn loops [1, 3].

2.2 The need for persistent, typed state

are-self is built around persistent, typed state. Almost every important runtime decision is driven by database rows and then executed asynchronously. This design enables inspectable state, audit trails, and recovery from failures without losing execution context [1].

2.3 Context: local, remote, and native execution

The GenericEffectorCaster routes each spike to either a native Python handler (e.g., `run_frontal_lobe`, `run_temporal_lobe`) or to NerveTerminal for local/remote process execution. NerveTerminal uses the same pipeline for both local and remote: async subprocess plus log tailing, yielding `NerveTerminalEvent` (T_LOG, T_EXIT). Remote execution is JSON-over-TCP [12, 1].

3 Method: are-self Architecture Overview

3.1 Premise: brain-themed modular decomposition

are-self decomposes functionality into modules named after brain regions. The CNS is the graph engine; the frontal lobe is the reasoning loop; the parietal lobe is the tool bridge; the hippocampus is long-term memory; the temporal lobe is the scheduler; the prefrontal cortex is the work gatekeeper; the identity module defines persona; the synaptic cleft is the event bus; the occipital lobe digests logs [2, 1].

3.2 Orchestration and cognition pillars

The architecture has two pillars: (i) *Orchestration*—CNS graph execution, PNS agent discovery and execution, environments for context—and (ii) *Cognition*—frontal lobe (reasoning), parietal lobe (tools), hippocampus (memory), temporal lobe (scheduling), prefrontal cortex (work allocation), identity (persona) [2].

3.3 Cross-module architecture

The combined documentation defines the cross-module flow: CNS connects to PNS and environments; CNS invokes frontal lobe and temporal lobe; frontal lobe connects to parietal lobe, hippocampus, and identity; temporal lobe connects to prefrontal cortex; prefrontal cortex invokes frontal lobe [2].

3.4 Two core methodological advances

Two core advances comprise are-self:

(A) Graph-based orchestration with labeled transition semantics and Celery-driven self-driving dispatch.

(B) Identity-driven autonomous reasoning with focus/XP economy, vector memory, and Agile work integration.

4 Method A: Central Nervous System and Graph Orchestration

4.1 Static graph model

The CNS static graph is $G = (V, E, \lambda)$ where V = Neurons, $E \subseteq V \times V$ = Axons, and $\lambda(e) \in \{\text{flow}, \text{success}, \text{failure}\}$. The runtime is an unfolding execution tree of Spikes, not a simple replay of G [3, 2].

4.2 Transition rule and wave dispatch

For finished spike s with $\sigma(s) \in \{\text{SUCCESS}, \text{FAILED}\}$:

$$L_{\text{enabled}}(s) = \{\text{flow}\} \cup \begin{cases} \{\text{success}\} & \sigma(s) = \text{SUCCESS} \\ \{\text{failure}\} & \sigma(s) = \text{FAILED} \end{cases}$$

For each $e = (\nu(s), v)$ with $\lambda(e) \in L_{\text{enabled}}(s)$: create spike at v. The `cast_cns_spell` Celery task executes one spike; in `finally`, it always calls `check_next_wave`, making the engine self-driving [3, 1].

4.3 Context resolution precedence

Context composition: $C = \text{metadata} \oplus \text{env} \oplus \text{blackboard} \oplus \text{effector} \oplus \text{neuron}$. Precedence (rightmost wins): neuron > effector > blackboard > env > metadata [3, 2].

4.4 Blackboard propagation and hierarchical composition

Child spikes copy the parent's blackboard. If a Neuron has `invoked_pathway`, are-self creates a child SpikeTrain; the parent spike becomes DELEGATED. On child completion, signals map the terminal status back to the parent [3, 1].

4.5 Distribution modes

After spike creation, are-self chooses cardinality: LOCAL_SERVER (1), ONE_AVAILABLE_AGENT (1 if online), ALL_ONLINE_AGENTS (n_{online}), SPECIFIC_TARGETS (n_{targets}) [3, 2].

5 Method B: The Reasoning Engine

5.1 Frontal Lobe: turn loop and prompt assembly

The frontal lobe orchestrates ReasoningSession and ReasoningTurn. Each turn assembles: (1) system identity prompt, (2) L1/L2 history cache, (3) volatile addon messages, (4) sensory trigger (engram catalog). L1: last 2 turns full; L2: older turns with truncated tool results and pruned monologue [4, 1].

5.2 Session economy

Session level and focus cap [4, 2]:

$$\text{current_level} = \lfloor \text{total_xp}/100 \rfloor + 1, \quad \text{max_focus} = 10 + \lfloor (\text{current_level} - 1) \cdot 0.5 \rfloor$$

Efficiency bonus: if $\text{len}(\text{thought_process}) \leq \text{current_level} \cdot 1000$, grant +1 focus and +5 XP [4].

5.3 Parietal Lobe: tool bridge and focus economy

The parietal lobe builds JSON schemas from `IdentityDisc.enabled_tools`, augments descriptions with `[COST: X Focus | REWARD: +Y XP]`, and executes tool calls sorted by `focus_modifier` descending. Fizzle: $\Delta f < 0 \wedge f + \Delta f < 0 \Rightarrow$ tool not executed [5, 2].

5.4 Hippocampus: vector memory and novelty rewards

Embedding $\phi : \mathcal{T} \to \mathbb{R}^{768}$; similarity $\text{sim}(\mathbf{a}, \mathbf{b}) = 1 - d_{\cos}(\mathbf{a}, \mathbf{b})$. Intercept: $\text{max_sim} \geq 0.90 \Rightarrow$ save rejected. Novelty: $\text{novelty} = \max(0, 1 - \text{similarity})$. Rewards: $\text{focus_yield} = \max(1, \lfloor 10 \cdot \text{novelty} \rfloor)$, $\text{xp_yield} = \max(5, \lfloor 100 \cdot \text{novelty} \rfloor)$ [6, 2].

5.5 Identity and addons

IdentityDisc provides system prompt, model, tools, addons. Addons (focus, deadline, agile) inject dynamic context per turn. The agile addon maps shifts to work instructions: SIFTING (PM refinement), EXECUTING (worker stories), SLEEPING (reflection) [9, 1].

5.6 Temporal Lobe and Prefrontal Cortex

Temporal Lobe: iteration state WAITING $\to$ RUNNING $\to$ FINISHED; participant SELECTED $\to$ ACTIVATED $\to$ COMPLETED. Prefrontal Cortex: $W(\text{shift}, \text{identity_type}, \text{disc}, \text{env})$ is a finite table. If no work, worker is stood down to SELECTED [7, 8, 2].

6 are-self Module Schema

Table 1 summarizes the are-self module structure [2, 1].

7 Supporting Systems

7.1 Synaptic Cleft: event taxonomy

$\mathcal{E}_{\text{synapse}} = \{\text{LOG}, \text{STATUS}, \text{BLACKBOARD}\}$. Glutamate $\mapsto$ LOG; Dopamine/Cortisol $\mapsto$ STATUS; Acetylcholine $\mapsto$ BLACKBOARD. Group: $G(s) =$ "spike_log_" $\oplus \text{str}(s)$ [10, 2].

7.2 Occipital Lobe: log budget

$\text{max_char_limit} = (\text{max_token_budget} - 2000) \times 4$. Error blocks: 5 lines before, 10 after; max 5 blocks [11, 2].

Table 1: are-self module schema.

Module	Key Structure	Purpose
central_nervous_system	$G = (V, E, \lambda)$, $L_{\text{enabled}}(s)$	Graph engine, wave dispatch, blackboard propagation
frontal_lobe	Session level, max_focus, efficiency bonus	AI reasoning loop, prompt assembly, L1/L2 cache
parietal_lobe	Tool schema, fizzle, focus/XP update	Tool bridge, ParietalMCP, cost/reward
prefrontal_cortex	$\mathcal{G}_{\text{PFC}}$, $W(\text{shift}, \text{type})$	Agile work graph, work-eligibility gatekeeper
temporal_lobe	Iteration state machine, shift sequence	Scheduler, metronome, Ouroboros
hippocampus	ϕ, cosine similarity, novelty, focus_yield	Durable memory, vector embeddings, intercept
identity	Addon composition, ADDON_REGISTRY	Persona, system prompt, addons
synaptic_cleft	$\mathcal{E}_{\text{synapse}}$, group $G(s)$	Websocket event bus, neurotransmitters
peripheral_nervous_system	NerveTerminal, discovery	Local/remote execution, agent discovery
environments	Single selection, VariableRenderer	Project context, variable interpolation
occipital_lobe	Error blocks, token budget	Log digestion, error extraction

7.3 Environments: context and selection

At most one ProjectEnvironment has `selected=True`. VariableRenderer uses Django template syntax for interpolation. Context precedence places env between metadata and blackboard [13].

8 State Machines and Invariants

Table 2 summarizes key state machines [2].

8.1 Invariants

From the combined documentation: (1) CNS: at most one Begin Play neuron per pathway; `cast_cns_spell` always calls `check_next_wave` in finally. (2) Environments: at most one ProjectEnvironment with selected=True. (3) Hippocampus: vector dim = 768; intercept threshold = 0.90. (4) PFC: no ReasoningSession without `_is_available_work` True. (5) Temporal: at most one active temporal SpikeTrain per environment [2].

9 Limitations and Assumptions

The graph is not guaranteed acyclic; retrigger prevention is via "has descendants" check. The prompt-window heuristic `num_ctx = floor(payload_size/3) + 2048` is not tokenizer-accurate.

Table 2: are-self state machines.

Entity	States	Terminal
Spike	CREATED, PENDING, RUNNING, SUCCESS, FAILED, ABORTED, DELEGATED, STOPPING, STOPPED	SUCCESS, FAILED, ABORTED, STOPPED
SpikeTrain	CREATED, RUNNING, SUCCESS, FAILED, STOPPING, STOPPED	SUCCESS, FAILED, STOPPED
ReasoningSession	PENDING, ACTIVE, PAUSED, COMPLETED, MAXED_OUT, ERROR, ATTENTION_REQUIRED, STOPPED	COMPLETED, MAXED_OUT, ERROR, STOPPED
Iteration	WAITING, RUNNING, FINISHED, CANCELLED, BLOCKED_BY_USER, ERROR	FINISHED, etc.
IterationShiftParticipant	SELECTED, ACTIVATED, COMPLETED, PAUSED, ERROR	COMPLETED, etc.

Occipital lobe uses 1 token $\approx$ 4 chars. Stale pending spikes ($>$ 5 min) are marked FAILED; zombie detection uses 20 minute cutoff [2, 3, 4, 7].

10 Conclusion

are-self is a Django control plane that unifies graph-based orchestration with identity-driven autonomous reasoning. The CNS executes labeled directed graphs as self-driving waves of Spikes, with formal transition semantics, blackboard propagation, and hierarchical subgraphs. The cognition stack implements a turn-based reasoning loop with focus/XP economy, vector memory with novelty rewards, and a work-eligibility gatekeeper that prevents spinning up AI workers without actionable work. The architecture is grounded in persistent, typed state and a brain-themed modular decomposition documented in the codebase and doc-generator [1, 2, 3, 4, 5, 6, 7, 8, 9].

References

[1] are-self Live Codebase Analysis. `docs/LIVE_CODEBASE_ANALYSIS.md`.

[2] are-self Backend Combined Mathematical Structure. `docs/doc-generator/combined__documentation.md`.

[3] Central Nervous System Documentation. `docs/doc-generator/central_nervous_system__documentation.m`

[4] Frontal Lobe Documentation. `docs/doc-generator/frontal_lobe__documentation.md`.

[5] Parietal Lobe Documentation. `docs/doc-generator/parietal_lobe__documentation.md`.

[6] Hippocampus Documentation. `docs/doc-generator/hippocampus__documentation.md`.

[7] Temporal Lobe Documentation. `docs/doc-generator/temporal_lobe__documentation.md`.

[8] Prefrontal Cortex Documentation. `docs/doc-generator/prefrontal_cortex__documentation.md`.

[9] Identity Documentation. `docs/doc-generator/identity__documentation.md`.

[10] Synaptic Cleft Documentation. `docs/doc-generator/synaptic_cleft__documentation.md`.

[11] Occipital Lobe Documentation. `docs/doc-generator/occipital_lobe__documentation.md`.

[12] Peripheral Nervous System Documentation. `docs/doc-generator/peripheral_nervous_system__documentat`

[13] Environments Documentation. `docs/doc-generator/environments__documentation.md`.

A CNS Transition Rule (Formal)

The enabled edge labels for finished spike s:

$$L_{\text{enabled}}(s) = \{\text{flow}\} \cup \begin{cases} \{\text{success}\} & \text{if } \sigma(s) = \text{SUCCESS} \\ \{\text{failure}\} & \text{if } \sigma(s) = \text{FAILED} \end{cases}$$

For each $e = (\nu(s), v)$ with $\lambda(e) \in L_{\text{enabled}}(s)$: create spike at v.

B Hippocampus Novelty and Rewards

When save is not intercepted:

$$\text{novelty} = \max(0, 1-\text{similarity}), \quad \text{focus_yield} = \max\left(1, \lfloor 10 \cdot \text{novelty} \rfloor\right), \quad \text{xp_yield} = \max\left(5, \lfloor 100 \cdot \text{novelty} \rfloor\right)$$